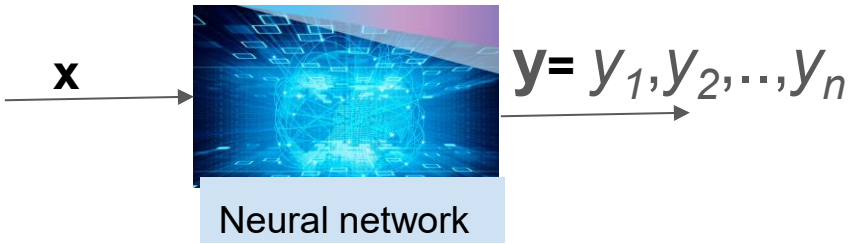
Gated RNNs

- Gates control which part of the long past is used for current prediction
- Gates also allow forgetting of part of the state
- LSTM: Long Short Term Memory, one of the most successful gated RNNs [Not in syllabus]
- An excellent introductions here:
  - http://colah.github.io/posts/2015-08-Understanding-LSTMs/
  - http://blog.echen.me/2017/05/30/exploring-lstms/
  - https://www.tensorflow.org/tutorials/text/text_generation

The sequence prediction task

- Given a complex input **x**
  - Example: sentence(s), image, audio wave
- Predict a sequence **y** of discrete tokens $y_1, y_2, .., y_n$
  - Typically a sequence of words.
  - A token can be any term from a huge discrete vocabulary
  - Tokens are inter-dependent
    - Not n independent scalar classification task.



Motivation

- Applicable in diverse domains spanning language, image, and speech processing.
- Before deep learning each community solved the task in their own silos → lot of domain expertise
- The promise of deep learning: as long as you have lots of labeled data, domain-specific representations learnable
- This has brought together these communities like never before!

Translation

Context: **x**

Predicted sequence: **y**

Where can I find healthy and traditional Indian food? →

स्वस्थ और पारंपरिक भारतीय भोजन कहां मिल सकता है?

- Pre-DL translation systems were driven by transfer grammar rules painstakingly developed by linguists and elaborate phrase translation
- Whereas, modern neural translation systems are scored almost 60% better than these domain-specific systems.

Image captioning

Context: **x**

Predicted sequence: **y**

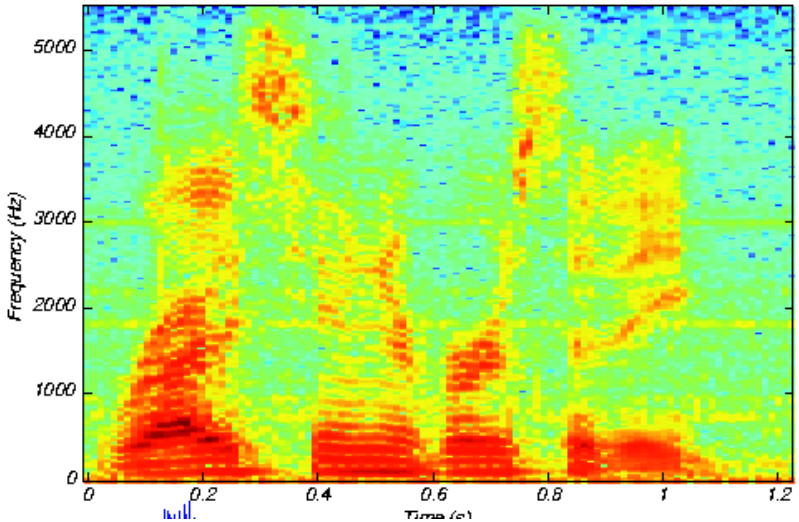
A person riding a motorcycle on a dirt road

- Early systems: either template-driven or transferred captions from related images
- Modern DL systems have significantly pushed the frontier on this task.

Image from http://idealog.co.nz/tech/2014/11/googles-latest-auto-captioning-experiment-and-its-deep-fascination-artificial-intelligence

Speech recognition

Context: **x (Speech spectrogram)**

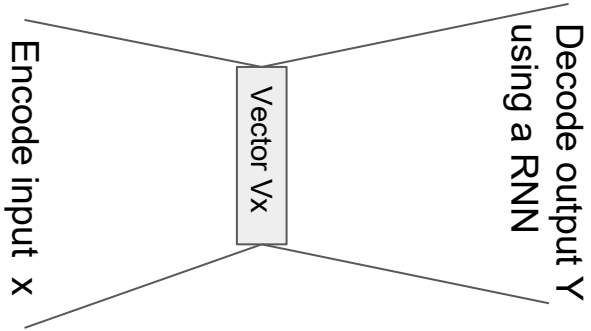


Output: **Y (Phoneme Sequence)**

Ri ce Uni ver si ty

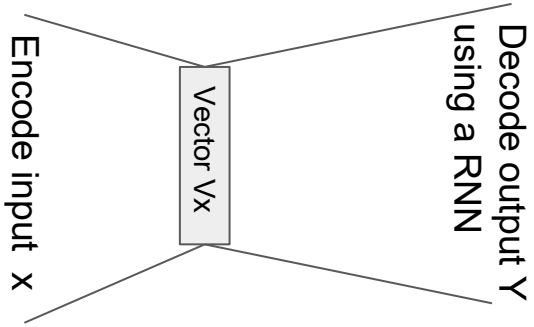
The Encoder Decoder model for sequence prediction

- Encode x into a fixed-D real vector X
- Decode y token by token using a RNN
  - Initialize a RNN state with X
  - Repeat until RNN generates a EOS token
    - Feed as input previously generated token
    - Get a distribution over output tokens, and choose best.



The Encoder Decoder model for sequence prediction

- Encode x into a fixed-D real vector X
- Since Y has many parts, need a graphical model to express the joint distribution over constituent tokens y1,...,yn. Specifically, we choose a special Bayesian network, called a RNN



$$P(\mathbf{y}|\mathbf{x},\theta) = \prod_{t=1}^{n} P(y_t|y_1,\ldots,y_{t-1},\mathbf{x},\theta)$$

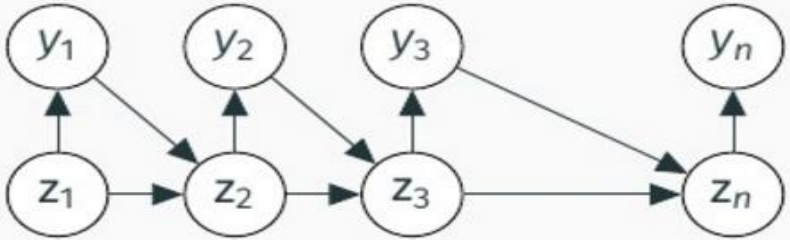
Encoder decoder model

- Let $\mathbf{v}_x$=fixed dimensional vector summary of input $\mathbf{x}$
- Difficult to define CPD $\Pr(y_t|y_1, \ldots, y_{t-1}, \mathbf{v}_x)$ with variable length of parents. Need to share parameters.
- Redesign the BN by summarizing parents as state.

$$\Pr(y_t|y_1, \ldots, y_{t-1}, \mathbf{v}_x, \theta) = P(y_t|\mathbf{z}_t, \theta), \tag{1}$$

where $\mathbf{z}_t$ is a state vector implemented using a recurrent neural network as

$$\mathbf{z}_t = \begin{cases} \mathbf{v}_x & \text{if } t = 0, \\ \text{RNN}(\mathbf{z}_{t-1}, y_{t-1}, \theta_R) & \text{otherwise.} \end{cases} \tag{2}$$



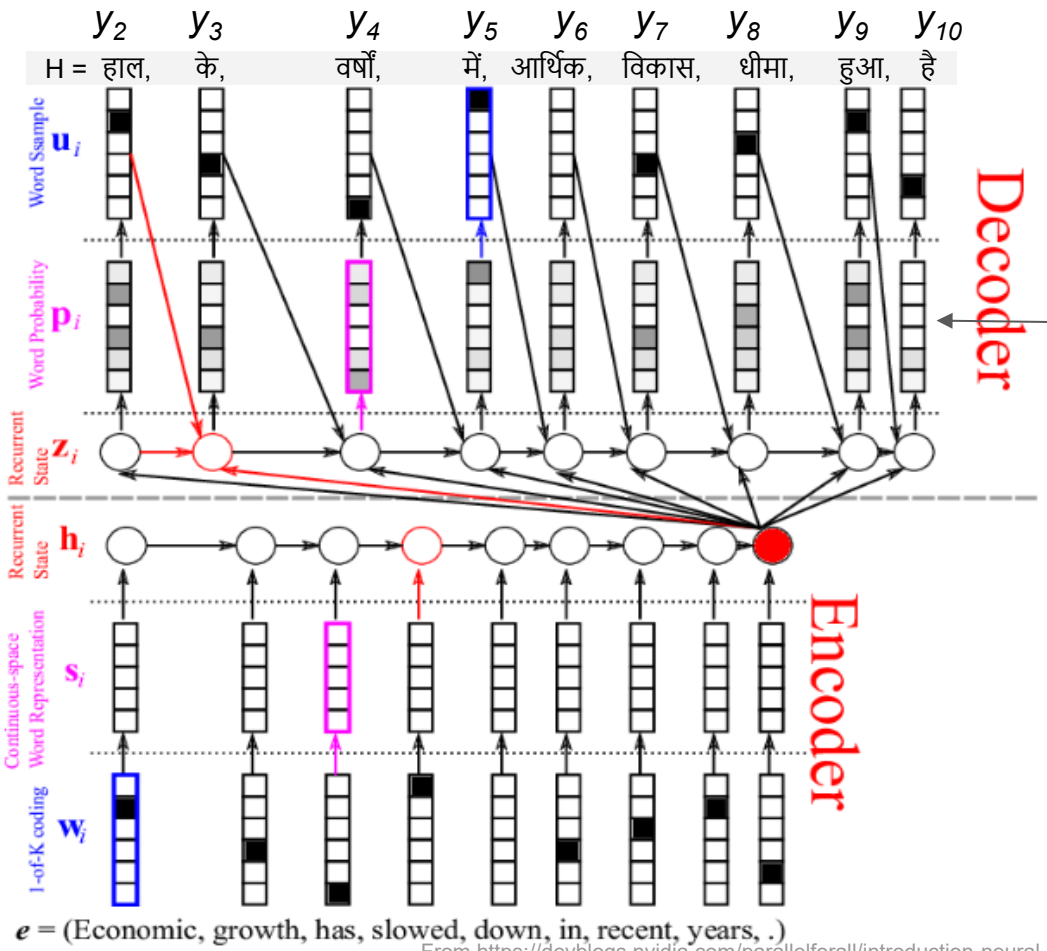
Encoder-decoder model

- Models full dependency among tokens in predicted sequence
  - Chain rule $P(\mathbf{y}|\mathbf{x},\theta) = \prod_{t=1}^{n} P(y_t|y_1,\ldots,y_{t-1},\mathbf{x},\theta)$
  - No conditional independencies assumed unlike in CRFs
- Training:
  - Maximize likelihood. Statistically sound!
- Inference

Inference

- Finding the sequence of tokens y1,....,yn for which product of probabilities is maximized
- Cannot find the exact MAP efficiently since fully connected Bayesian network ⇒ intractable junction tree. The states z are high-dimensional real-vectors.
- Solution: approximate inference
  - Greedy
  - Beam-search: branch & bound expansion of frontier of ‘beam width’
    - Probability of predicted sequence increases with increasing beam width.

Encoder-decoder for sequence to sequence learning



Predicted sequence: **y**

Choose high probability token and feed to next step.

$$\Pr(y_t | y_1, \ldots, y_{t-1}, \mathbf{x})$$

RNN to generate **y**

RNN e.g. LSTMs to summarize x token-by-token

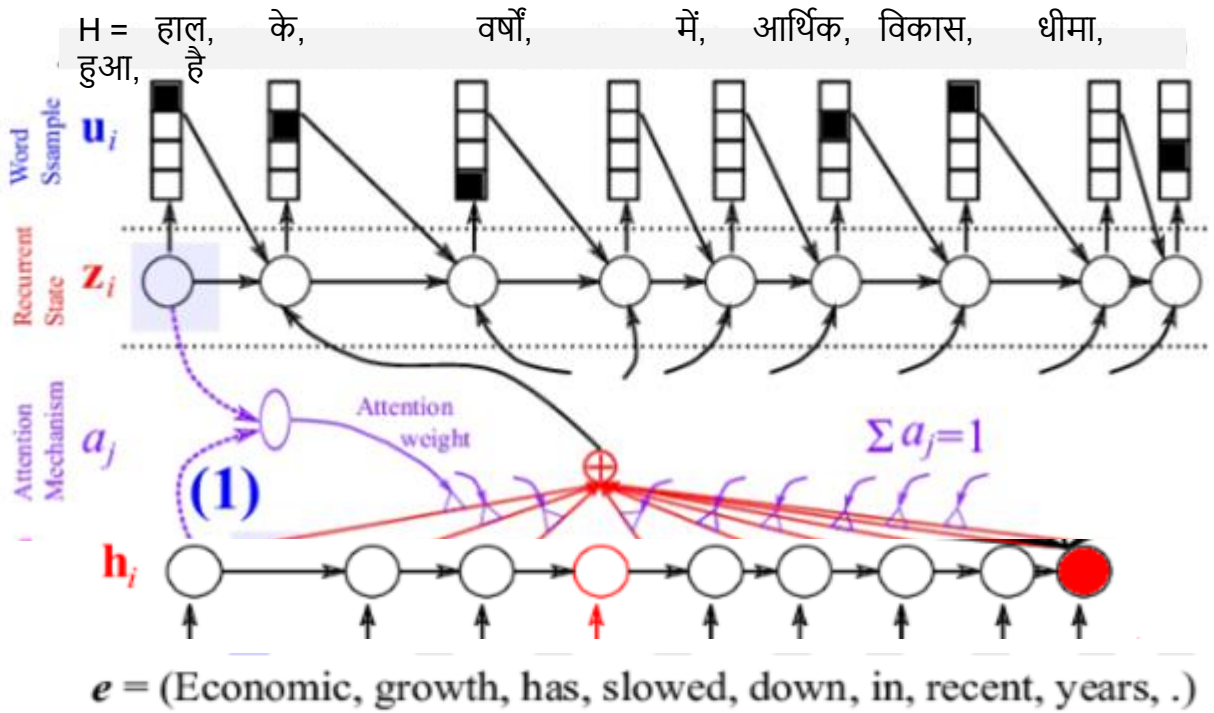
Embedding layer to convert each word to a fixed-D real vector

Context: **x**

From https://devblogs.nvidia.com/parallelforall/introduction-neural-machine-translation-gpus-part-2/

Single vector not powerful enough ---> revisit input

Deep learning term for this ⇒ Attention!



How to learn attention automatically, and in a domain neutral manner?

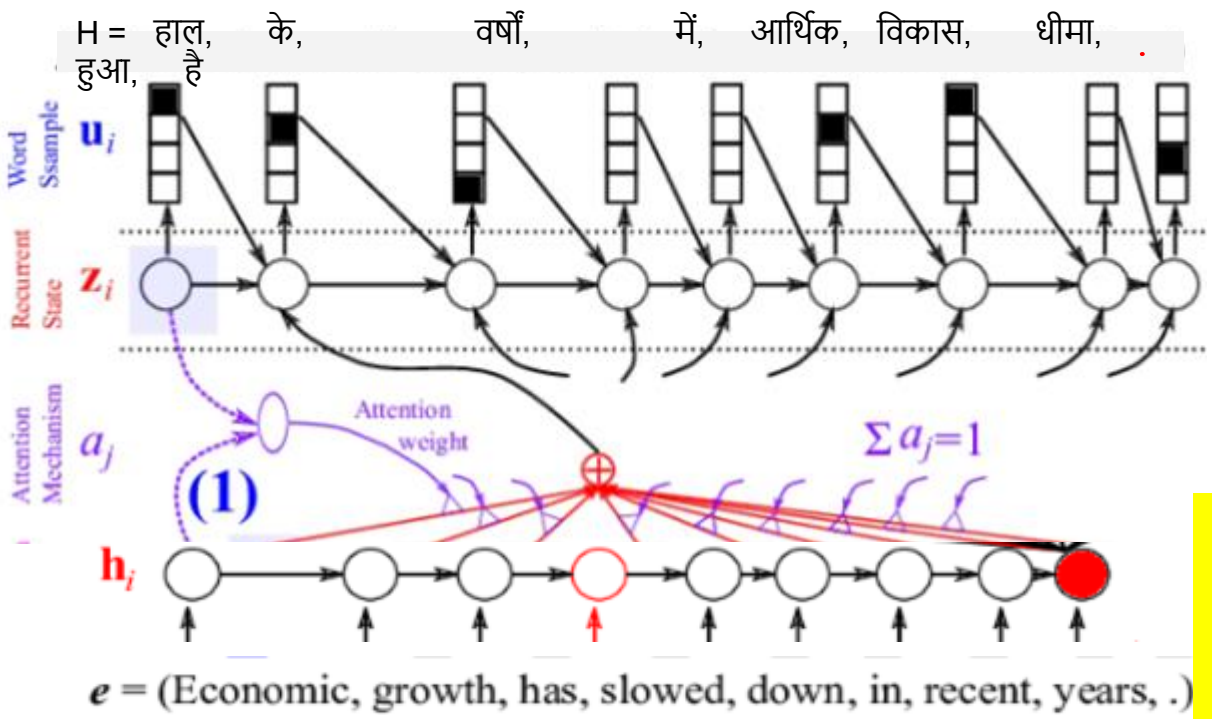
From https://devblogs.nvidia.com/parallelforall/introduction-neural-machine-translation-gpus-part-2/

Single vector not powerful enough ---> revisit input

Deep learning term for this ⇒ Attention!

Some examples of
$Attn(z, h) = z^T h$ (1)

$FF_\psi(z, h) \rightarrow R$ (2)



$$Attn(z_{t-1}, h_i) = A_{ti}$$

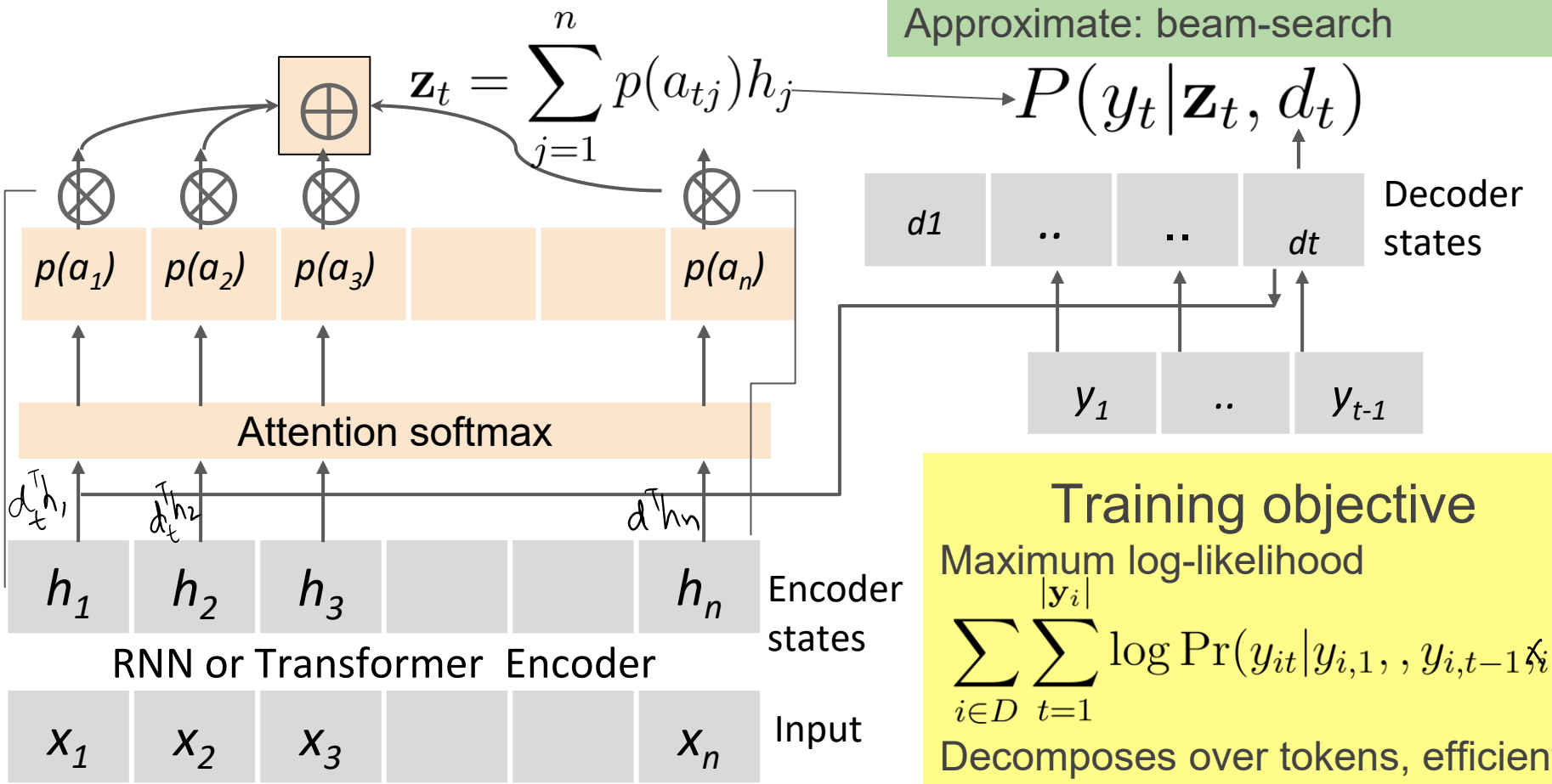
$$\alpha_{ti} = \frac{\exp(A_{ti})}{\sum_p \exp(A_{tp})}$$

$$v_{x,t} = \sum_i \alpha_{ti} h_i$$

End-to-end trained and magically learns to align automatically given enough labeled data

From https://devblogs.nvidia.com/paralle...

Encoder-Decoder (ED) Model



Inference

Exact: intractable.

Approximate: beam-search

Training objective

Maximum log-likelihood

$$\sum_{i \in D} \sum_{t=1}^{|\mathbf{y}_i|} \log \Pr(y_{it} | y_{i,1}, , y_{i,t-1}, \mathbf{x}_i)$$

Decomposes over tokens, efficient!

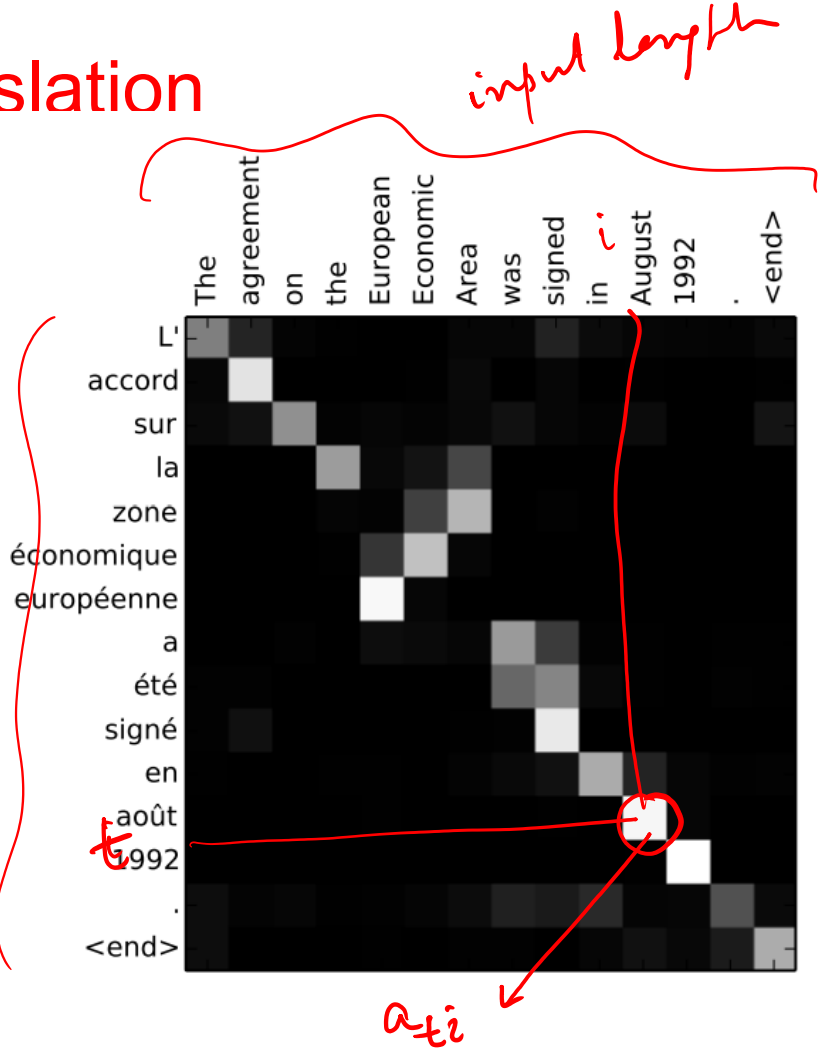
Example of attention in translation

Nice animated explanations for attention.

https://distill.pub/2016/augmented-rnns/#attentional-interfaces

https://www.tensorflow.org/tutorials/text/nmt_with_attention

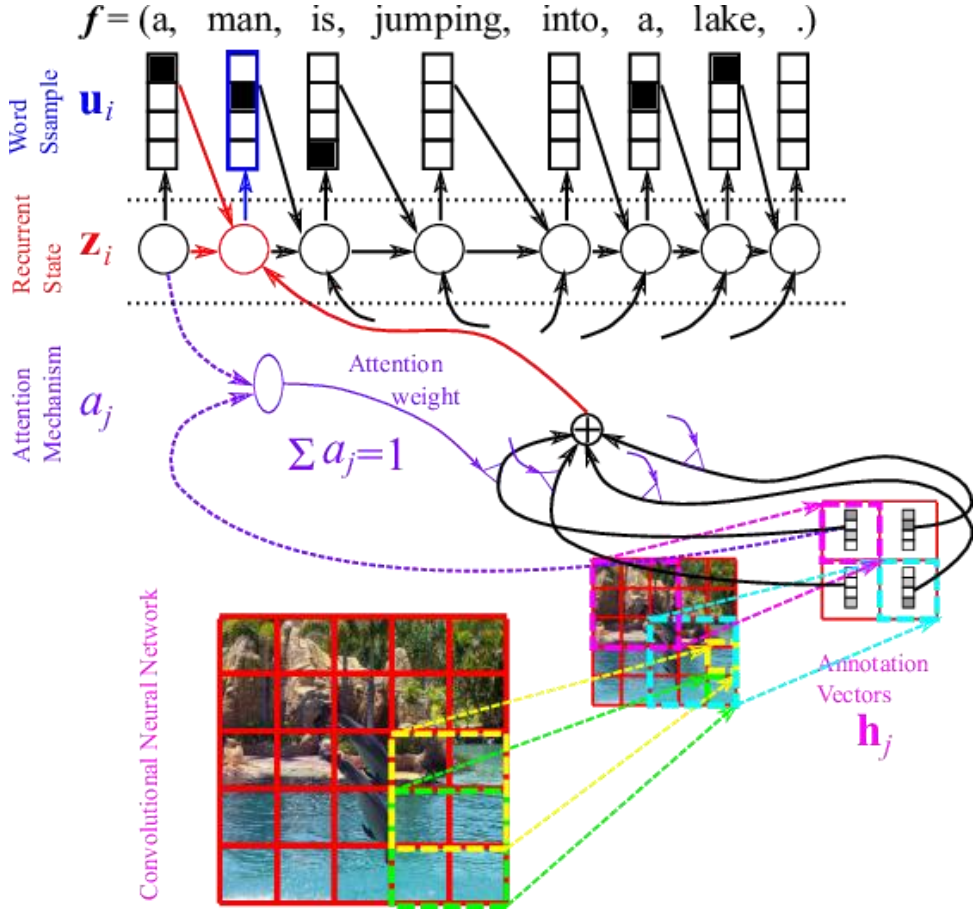
https://colab.research.google.com/github/tensorflow/text/blob/master/docs/tutorials/nmt_with_attention.ipynb



Attention

- Attention
  - Visualizing A Neural Machine Translation Model (Mechanics of Seq2seq Models With Attention) – Jay Alammar – Visualizing machine learning one concept at a time. (jalammar.github.io)

Same attention logic applies to other domains too



Attention over CNN-derived features of different regions of image

From https://devblogs.nvidia.com/parallelforall/introduction-neural-machine-translation-gpus-part-2/

Attention in image captioning. Attention over CNN

A bird flying over a body of water.

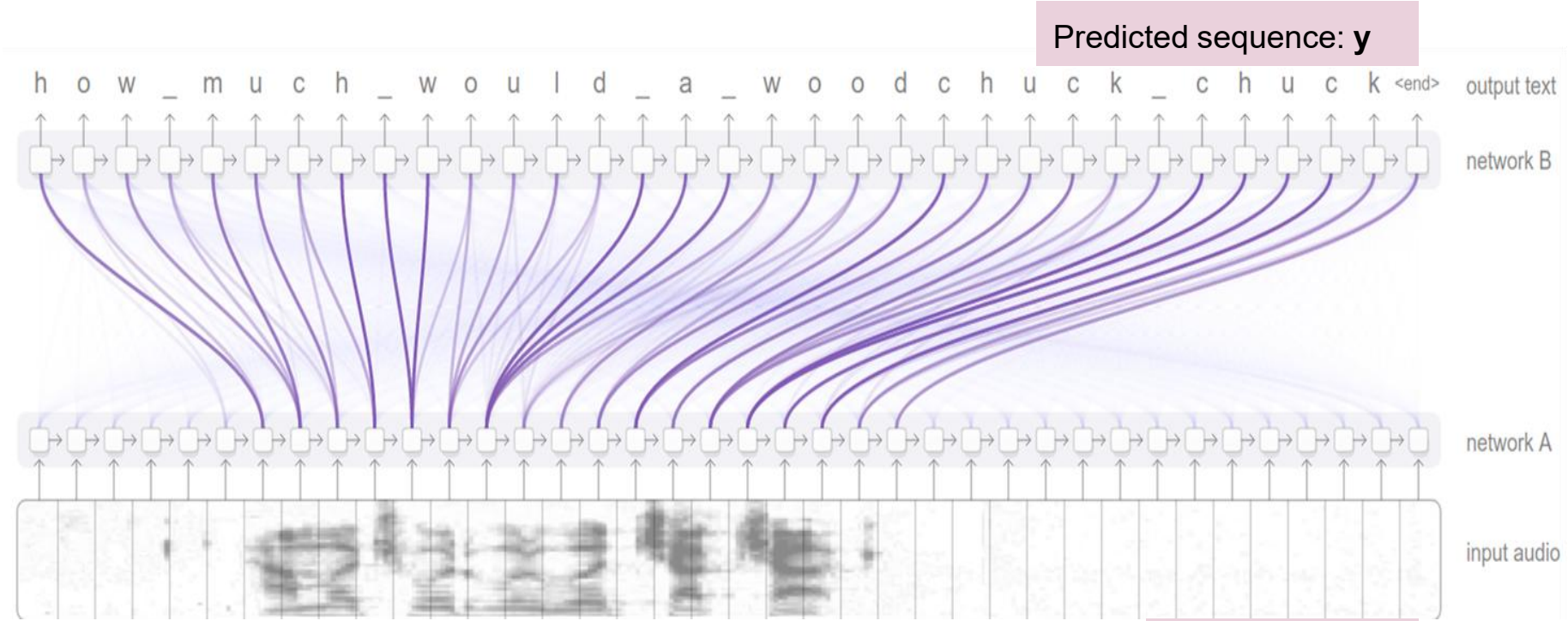
A woman is throwing a frisbee in a park.

A dog is standing on a hardwood floor.

A stop sign is on a road with a

From https://arxiv.org/pdf/1502.03044v3.pdf

Attention in Speech to Text Models



We see that attention is focussed in middle part and nicely skips the prefix and suffix that is silence.

Diagram from https://distill.pub/2016/augmented-rnns/

Transformers

- RNNs require sequential computation of state, which makes training slow
- Transformers replace “state” with multi-layered self-attention.
- All positions in a sequence can be trained in parallel.

- https://jalammar.github.io/illustrated-transformer/

Position embeddings.

- Goal: For each word position, create a unique vector of size d, that satisfies these properties:
  - Each position has a unique vector
  - New-by positions have similar vectors
  - Support a large number of positions (say C, without requiring parameter retraining even if sentences of length C have not been seen during training)

Options

Integer positions: Assign each position a integer id and pad

- Limitation: Neural networks cannot handle large integers as inputs.

Learned position embeddings: Like word embeddings, learn an embedding for each position.

- Limitation: Cannot handle positions beyond those seen during training.

Options: Binary encoding of position id

- Limitation: Binary vectors of adjacent positions may not be necessarily close together.
- Cannot easily capture new distances.

Options: Sinosoidal position embeddings.

- Position embedding of i-th token

$$p_{i,2j} = \sin\left(\frac{i}{C^{2j/d}}\right), \; p_{i,2j+1} = \cos\left(\frac{i}{C^{2j/d}}\right)$$

- Example: for d=4

$$\boldsymbol{p}_i = [\sin(\frac{i}{C^{0/4}}), \cos(\frac{i}{C^{0/4}}), \sin(\frac{i}{C^{2/4}}), \cos(\frac{i}{C^{2/4}})]$$

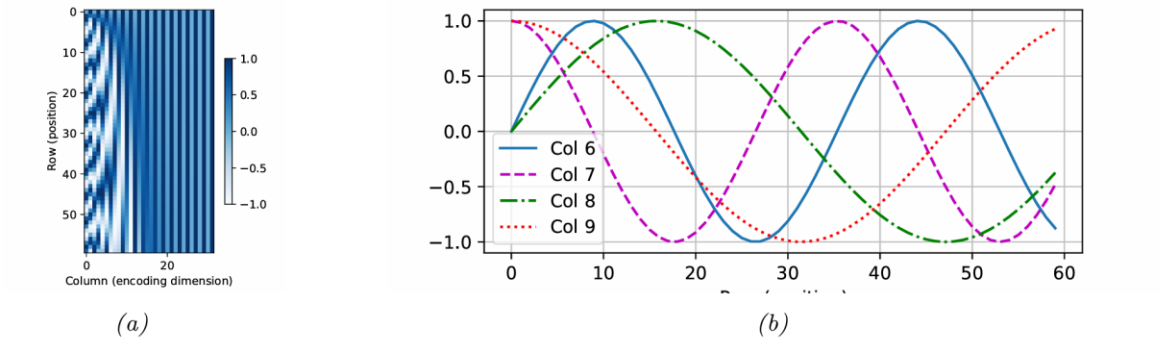


Figure 15.25: (a) Positional encoding matrix for a sequence of length $n = 60$ and an embedding dimension of size $d = 32$. (b) Basis functions for columns 6 to 9. Generated by positional_encoding_jax.ipynb.

Nearby positions are close together in this space.

- Consider

$$\begin{pmatrix} \sin(\omega_k(t+\phi)) \\ \cos(\omega_k(t+\phi)) \end{pmatrix} = \begin{pmatrix} \sin(\omega_k t)\cos(\omega_k \phi) + \cos(\omega_k t)\sin(\omega_k \phi) \\ \cos(\omega_k t)\cos(\omega_k \phi) - \sin(\omega t)\sin(\omega_k \phi) \end{pmatrix}$$

$$= \begin{pmatrix} \cos(\omega_k \phi) & \sin(\omega_k \phi) \\ -\sin(\omega_k \phi) & \cos(\omega_k \phi) \end{pmatrix} \begin{pmatrix} \sin(\omega_k t) \\ \cos(\omega_k t) \end{pmatrix}$$